

Dokumentation der angepassten Docker- OTOBO-Umgebung für HiSolutions

© 2025 maxence business consulting gmbh, <https://www.maxence.de/>

GNU AFFERO GENERAL PUBLIC LICENSE Version 3, November 2007

This work is copyrighted by
maxence business consulting gmbh
Am Weißen Stein 1
41541 Dormagen
Germany

Inhalt

1.0	Allgemein	3
2.0	Beschreibung	3
3.0	Voraussetzungen	3
4.0	Installation OTOBO	5
5.0	Installation html2pdf.....	9
6.0	Kontakt.....	12

1.0 Allgemein

Dies ist eine erweiterte Version der OTOBO-Docker-Installation, mit aktuellen OS-Grundlagen der Container sowie Vorgaben für die Konfiguration.

Bei Fragen zum Paket wenden Sie sich bitte an das maxence Support Team (otobo-support@maxence.de).

2.0 Beschreibung

Die Installation besteht aus 5 Images:

- Service db: MariaDB ist das Standard-Datenbanksystem.
- Service elastic: Elasticsearch beschleunigt bei vielen Tickets/Artikeln die Suche in OTOBO.
- Service redis: Redis für schnelleres Caching.
- Service web: Gazelle ist der Webserver für die Verarbeitung des Perl-Codes.
- Service nginx: Nginx wird als Reverse Proxy für die HTTPS-Unterstützung benötigt.

Außerdem ein Dockerfile zum Bau eines aktualisierten html2playwright-Images.

3.0 Voraussetzungen

- Docker ab Version 19.03.13
- Docker Compose ab Version 1.25.0, empfohlen ist Docker V2 spätestens ab Ubuntu 24.04 LTS
-> Diese Anleitung bezieht sich auf Docker V2, d.h. dass die Befehle nicht mehr `docker-compose` sondern `docker compose` lauten. Entsprechend müssen in den nachfolgenden Update-Befehlen ggf. wieder Bindestriche eingefügt werden.
- WICHTIG!
Docker muss aus dem Docker-Repo installiert werden (<https://docs.docker.com/engine/install/ubuntu/#install-using-the-repository>), nicht aus den Ubuntu-Paketquellen. Letzteres installiert Docker als Snap, welches eine eigene Abschottung mitbringt und so das Einbinden von Volumes in Container zuverlässig verhindert.
- Eine passende Kerberos-Konfiguration, mit den entsprechenden Dateien `krb5.conf` und `krb5.keytab`

4.0 Installation Docker-OTOBO

Das Archiv nach /opt entpacken, es muss sich eine Struktur /opt/otobo-docker/... ergeben. Idealerweise kopiert man die beiden krb5-Dateien in das Verzeichnis otobo-docker/nginx-conf/, dann müssen die beiden Zeilen im nächsten Schritt in der .env Datei nicht angepasst werden.

Das Compose-File .docker_compose_env_https_kerberos-HiS in .env umbenennen:

```
cp -p .docker_compose_env_https_kerberos-HiS .env
```

Anschließend die .env-Datei anpassen, die entsprechenden Zeilen sind mit dem Kommentar „HiSolutions“ versehen.

In dieser Datei wird ein OTOBO_DB_ROOT_PASSWORD gesetzt, dieses wird später noch mal für den grafischen Installationsvorgang benötigt.

Benötigte persistente Volumes und Konfigurationsdateien erstellen

SSL:

Zertifikat und -Keyfile über einen Mountpoint eingebunden.

Zuerst wird ein Volume erstellt:

```
docker volume create otobo_nginx_ssl
```

```
otobo_nginx_ssl_mp=$(docker volume inspect --format '{{.Mountpoint}}' otobo_nginx_ssl)
```

```
echo $otobo_nginx_ssl_mp # nur zur Erfolgskontrolle
```

Dann werden Zertifikat und Keyfile in das Volume kopiert:

```
cp /Pfad/zu/ssl-cert.crt /Pfad/zu/ssl-key.key $otobo_nginx_ssl_mp
```

```
ll $otobo_nginx_ssl_mp # nur zur Erfolgskontrolle
```

Die Pfadkonfiguration für die SSL-Dateien ist im Image hart auf '/etc/nginx/ssl/' codiert und kann nicht geändert werden. In .env-Datei dürfen deswegen nur die Dateinamen angepasst werden, nicht die Pfade davor!

Nginx:

Es gibt kein Default-Konfigurations-File, nur ein Template, das per Script aufbereitet wird.

Deswegen muss das Templatefile manuell ergänzt werden.

Als Erstes wird wieder ein Volume erstellt...

```
docker volume create otobo_nginx_custom_config
```

```
otobo_nginx_custom_config_mp=$(docker volume inspect --format '{{ .Mountpoint }}'
otobo_nginx_custom_config)
```

```
echo $otobo_nginx_custom_config_mp # nur zur Erfolgskontrolle
```

...und die default-Konfiguration in's neue Volume kopiert:

```
docker create --name tmp-nginx-container rotheross/otobo-nginx-webproxy:latest-
11_0
```

```
docker cp tmp-nginx-
container:/etc/nginx/templates/otobo_nginx.conf.template
$otobo_nginx_custom_config_mp # benötigt eventuell 'sudo'
```

```
ls -l $otobo_nginx_custom_config_mp/otobo_nginx.conf.template # nur zur
Erfolgskontrolle, benötigt eventuell 'sudo'
```

```
docker rm tmp-nginx-container # der wird nicht mehr benötigt
```

Nun kann man die nginx-Konfiguration erweitern, in dem man in der ,location'-Klammer weitere Header konfiguriert:

```
vim $otobo_nginx_custom_config_mp/otobo_nginx.conf.template
```

(alternativ direkt im File:

```
vim /var/lib/docker/volumes/otobo_nginx_custom_config/_data/
otobo_nginx.conf.template
)
```

Einfach vor der schließenden Klammer des location-Blocks einfügen:

```
add_header Strict-Transport-Security 'max-age=31536000; includeSubDomains;
preload';
```

```
add_header X-XSS-Protection "1; mode=block";
```

```
add_header X-Content-Type-Options "nosniff" always;
```

```
add_header Referrer-Policy "strict-origin";
```

Jetzt kann das ganze Konstrukt mit

```
docker compose up -detach
```

gestartet werden.

Nach ein paar Sekunden kann das Ergebnis mit

```
docker ps
```

kontrolliert werden. otobo-daemon-1 und otobo-web-1 laufen dabei ggf noch nicht rund, da die Datenbank noch nicht konfiguriert ist.

Dafür wird OTOBO-Installer.pl ausgeführt, hiermit wird die Basis-Konfiguration von OTOBO gesetzt. Der Installer wird mit über die URL <https://FQDN/otobo/installer.pl> aufgerufen. Der Installer ist soweit selbsterklärend. Dazu die Anmerkungen:

- Bei der Datenbankprüfung wird das OTOBO_DB_ROOT_PASSWORD benötigt, das in der .env gesetzt wurde (s.o.)
- Die Einstellungen in "Allgemeine Einstellungen" können im Nachhinein in der SysConfig geändert werden, sollten aber idealerweise gleich "korrekt" vorgenommen werden.
- Auf jeden Fall sollte der http-Typ auf "https" gestellt werden, und das Protokollmodul auf "SysLog"
- Der Schritt mit der Mailkonfiguration kann vorerst übersprungen werden, die Konfiguration kann später in der SysConfig nachgeholt werden

Ist der Installer erfolgreich durchgelaufen und das Frontend erreichbar, müssen die Container wieder heruntergefahren werden, um die von OTOBO geschriebene Config.pm für SSO anpassen zu können:

```
docker compose down
```

```
vi /var/lib/docker/volumes/otobo_opt_otobo/_data/Kernel/Config.pm
```

In den Block 'insert your own config settings "here"' (prinzipiell egal, aber wenn es schon einen solchen Block gibt) die Zeilen

```
$Self->{'Frontend::AvatarEngine'} = 'None';  
$Self->{'AuthModule'} = 'Kernel::System::Auth::HTTPBasicAuth';  
$Self->{'LogModule'} = 'Kernel::System::Log::SysLog';
```

einfügen.

Bei Bedarf können hier weitere Konfigurationen wie z.B. die AD-Anbindung für Agenten-

und Kundendaten vorgenommen werden (siehe LDAP-Anbindungen.pm im Dokumentationsordner).

Nun können die Container mit

```
docker compose up -detach
```

wieder gestartet werden, der Login per SSO sollte nun funktionieren.

(Hier müssen ggf im Browser noch die Einstellungen für

`network.negotiate-auth.delegation-uris`

und

`network.negotiate-auth.trusted-uris`

an die Domäne angepasst werden)

5.0 Update Docker-OTOBO

Ein Backup ist immer angeraten!

Die Container stoppen mit

```
cd /opt/otobo-docker
```

```
docker compose down
```

5.1 Zwischenschritt: Update auf 10.1

Der Zwischenschritt kann noch aus den Rother-Repositories erfolgen

5.1.1 Automatisches Update per Skript

Laut offizieller Anleitung kann man mit dem Skript

```
./scripts/update.sh
```

und dem separat auszuführenden DB-Update-Skript

```
docker exec -it otobo-web-1 perl scripts/DBUpdate-to-10.1.pl
```

das Update weitestgehend automatisieren. Dieses Skript funktionierte bei mir (bzw. anderen Kunden) nicht immer zuverlässig, den Versuch ist es aber Wert, zerschossen wurde dabei auch nie etwas. (Und für das Update auf 11 ist das Skript auch nur noch für Minor-Updates angegeben)

Falls das Skript nicht zum gewünschten Erfolg führt:

5.1.2 Update von Hand

```
git fetch --tags          # 10_1_16 sollte das jüngste sein
```

```
git checkout rel-10_1_16
```

Anpassung/Ergänzung der bestehenden .env-Datei mit folgender Zeile:

```
OTOBO_IMAGE_OTOBO=rotheross/otobo:rel-10_1_16
```

Und mit

```
docker compose pull
```

das neue Release ziehen, dann das OTOBO-Release kopieren

```
docker compose run --no-deps --rm web copy_otobo_next
```

Jetzt können die Container wieder hochgefahren werden

```
docker compose up --detach
```

(und zur Kontrolle `docker ps`)

Dann das Skript für die OTOBO-Updates ausführen

```
docker compose exec web /opt/otobo_install/entrypoint.sh do_update_tasks
```

und da es sich um ein Major-Update handelt, auch noch das Datenbank-Update-Skript

```
docker exec -it otobo-web-1 perl scripts/DBUpdate-to-10.1.pl
```

Ein Blick auf das Update-Log sollte zeigen, dass alles nach Plan lief:

```
docker compose exec web cat /opt/otobo/var/log/update.log
```

5.2 Update auf 11.0

Das aktualisierte Zip-File muss in's vorhandene Installationsverzeichnis entpackt werden, dann müssen die Datei `.docker_compose_env_https_kerberos_HiS` und die vorhandene `.env` in der Datei `.env` zusammengeführt werden.

Im Wesentlichen dürfte das die Variablen für `OTOBO_DB_ROOT_PASSWORD` und die beiden SSL-Dateinamen aus der alten `.env` betreffen. Darüber hinaus müssen für das SSO die Kerberos-Variablen gefüllt werden.

Danach gibt es wieder 2 Möglichkeiten zum Update:

- 5.2.1 Für reine Patchlevel-Updates, also innerhalb eines Major-Release (z.B. 11.0.x) kann man wieder das Update-Skript nutzen:

```
./scripts/update.sh
```

Dieses Skript funktionierte bei mir (bzw anderen Kunden) nicht immer zuverlässig, es hat aber in Verbindung mit dem DB-Update-Skript auch schon bei einem

Major-Update auf 11 funktioniert, obwohl nicht explizit dafür angegeben.

5.2.2 Von Hand:

Mit

```
docker compose pull
```

werden die aktuellen Image-Dateien gezogen und

```
docker compose run --no-deps --rm web copy_otobo_next
```

kopiert die neue OTOBO-Version, die angepassten Dateien in den Volumes sollten davon nicht beeinträchtigt werden.

Danach können die Container wieder gestartet werden:

```
docker compose up --detach
```

Jetzt müssen noch die Update-Skripte ausgeführt werden

```
docker compose exec web /opt/otobo_install/entrypoint.sh  
do_update_tasks
```

Auch hier muss in beiden Fällen noch das DB-Update-Skript ausgeführt werden:

```
docker compose exec web perl scripts/DBUpdate-to-11.0.pl
```

Wenn die CMDB-Erweiterung installiert ist, muss diese gesondert geupdatet werden, da hier mit OTOBO 11 eine umfassende Umstellung auf dynamische Felder stattfand:

```
docker exec -it otobo_web_1 perl bin/otobo.Console.pl  
Admin::ITSM::Configitem::UpgradeTo11
```

6.0 Kontakt

Dieses Add-on wurde erstellt von der

maxence business consulting gmbh
An Weißen Stein 1
41541 Dormagen
Tel: +49 2133 2599-0
Mail: info@maxence.de

<https://www.maxence.de/>

maxence ist ein etabliertes IT-Beratungshaus, das seine Kunden seit mehr als 20 Jahren bei der Schaffung von Service Management Organisationen unterstützt und Anwendungs-lösungen dafür plant und entwickelt.

Herstellerunabhängige Beratung bieten wir u.a. bei der Implementierung eines IT Service Managements (ITSM) nach ITIL, bei der Optimierung von Prozessen und deren IT-gestützter Umsetzung sowie dem Aufbau eines Projekt Portfolio Managements oder Projekt Management Offices (PMO). Weiter bieten wir Beratung und Lösungen für die Bereiche IT-Helpdesk, Consumer-Support und Personalwesen an.

Technisch liegt unser Schwerpunkt auf Open Source Lösungen, die wir nahtlos in die Systemlandschaft unserer Kunden einbetten. Als ehemaliges Team der Lotus Professional Services beraten wir zudem umfassend zu HCL Notes/Domino und realisieren Anwendungen und Integrationen für diese hochflexible Plattform.

maxence Kunden sind Konzerne in der Energieversorgung sowie der chemisch-pharmazeutischen Industrie, Unternehmen in der Telekommunikation und Breitband-Versorgung, öffentliche Auftraggeber, Banken und Versicherungen sowie kleine und mittelständische Unternehmen unterschiedlichster Branchen.